

```
usinglong binary_search(const Type *f, usinglong n, const type z)
```

```
{
    usinglong nlo=0, nmi=n-1;
    while ( nlo != nmi )
    {
        usinglong t = (nmi+nlo)/2;
        if ( f[t] < z ) nlo = t + 1;
        else          nmi = t;
    }
    if ( f[nmi]==z ) return nmi;
    else          return ~OUL;
}
```

The above code works by the subdivision of the data, and it must not have n equal to zero.

To quantify it, consider an array of floating point numbers that are equally distributed in a given interval [with $\min(v)$ and $\max(v)$]. If the specific value here equals v , we can go for:

$$\frac{n - \min(n)}{\max(n) - \min(n)} = \frac{v - \min(v)}{\max(v) - \min(v)}$$

Here n represents the index, and $\min(v)$ and $\max(v)$ represent the index limits. We can obtain an equation for n as:

$$n = \min(n) + \frac{\max(n) - \min(n)}{\max(v) - \min(v)} (v - \min(v))$$

This relation will be useful while writing the related algorithms. Just as with the previous case, we can go for an approximated search, as shown below:

```
usinglong binarysearch_approx(const type *f, usinglong n, const type v, type da)
```

```
{
    usinglonglong p = binarysearch_ge(f, n, v-da);
    if ( p < n ) p = binarysearch_le(f+p, n-p, v+da);
    return p;
}
```

The code will have significance only if it meddles with the float or double types.

Shell codes

We are going to discuss some more shell scripts that might be useful if you want to add extra features to your application (based on a custom algorithm). These scripts essentially beautify your mother program.

Here is a code that helps you to schedule events:

```
#!/bin/sh
```

```
schedule_file="$HOME/.schedule"
```

```
check_day_name()
{
    case $(echo $1 | tr '[:upper:]' '[:lower:]') in
        jan*|feb*|mar*|apr*|may*|jun*) return 0 ;;
        sun*|mon*|tue*|wed*|thu*|fri*|sat*) retval=0 ;;
        *) retval=1 ;;
    esac
    return $retval
}

check_month_name()
{
    case $(echo $1 | tr '[:upper:]' '[:lower:]') in
        jan*|feb*|mar*|apr*|may*|jun*) return 0 ;;
        jul*|aug*|sep*|oct*|nov*|dec*) return 0 ;;
        *) return 1 ;;
    esac
}

normalize()
{
    echo -n $1 | cut -c1 | tr '[:lower:]' '[:upper:]'
    echo $1 | cut -c2-3 | tr '[:upper:]' '[:lower:]'
}
```

```
if [ ! -w $HOME ]; then
    echo "$0: The script can't write in your home directory ($HOME)">&2
    exit 1
fi
```

```
echo "Schedule: Voyage to Kernel Code"
echo -n "Date of event (in proper format): "
read word1 word2 word3 junk
```

```
if check_day_name $word1; then
    if [ ! -z "$word2" ]; then
        echo "Invalid dayname format: please specify the day name">&2
        exit 1
    fi
    date="$(normalize $word1)"
else
```

```

    if [ -z "$word2" ]; then
        echo "Invalid dayname format: unknown day name specified">&2
        exit 1
    fi
```

```
if [ ! -z "$(echo $word1 | sed 's/[[:digit:]]//g')" ]; then
    echo "Invalid date format: please specify day first, by day number">&2
    exit 1
fi
```

```
if [ "$word1" -lt 1 -o "$word1" -gt 31 ]; then
    echo "Invalid date format: day number can only be in range 1-31">&2
    exit 1
fi
```